



**Politechnika
Śląska**

**Wydział Automatyki, Elektroniki
i Informatyki**

Komunikacja i Nawigacja w Systemach Mobilnych

Interfejs diagnostyczny OBD2 z połączeniem
bezprzewodowym i wyświetlaczem

Kacper Stiborski, Maciej Lower

Gliwice 2026

Interfejs diagnostyczny OBD2 z połączeniem bezprzewodowym i wyświetlaczem

Streszczenie

Projekt zakłada wykonanie dwóch stacji bazujących na mikrokontrolerach umożliwiających zestawienia połączenia Bluetooth. Jedna ze stacji, wpinana w gniazdo diagnostyczne OBD2 odczytuje dane z magistrali CAN i przesyła je transmisją bezprzewodową do drugiej stacji z wyświetlaczem i interfejsem użytkownika. Przesyłane i analizowane będą dane podstawowe, informacyjne jak: prędkość samochodu i spalanie paliwa. Po zbadaniu danych z magistrali CAN możliwe będzie rozszerzenie o dodatkowe informacje np. diagnostyczne. Planowana jest topologia połączenia BLE: skaner OBD2 jako serwer (BLE Peripheral) oraz wyświetlacz z interfejsem użytkownika jako klient (BLE central). Wykorzystane zostaną do obsługi połączenia mikrokontrolery z rodziny Espressif, ponieważ posiadają wbudowaną obsługę BLE.

Słowa kluczowe

Elektornika, Pomiar, Komunikacja mobilna, CAN, BLE

Development of a research module enabling communication via LoRa

Abstract

The project involves the development of two stations based on microcontrollers enabling a Bluetooth connection. One of the stations, plugged into the OBD2 diagnostic port, reads data from the CAN bus and transmits it wirelessly to the second station, which features a display and a user interface. The transmitted and analyzed data will include basic informational parameters, such as vehicle speed and fuel consumption. After studying the data from the CAN bus, it will be possible to expand the system with additional information, e.g., diagnostic data. The planned BLE topology will consist of the OBD2 scanner acting as a server (BLE Peripheral), and the display with the user interface acting as a client (BLE Central). Microcontrollers from the Espressif family will be used to manage the connection, as they have built-in BLE support.

Key words

Electronics, Measurements, Mobile communication, CAN, BLE

Spis treści

1	Wstęp, cel i założenia projektu	3
1.1	Wprowadzenie w zagadnienie	3
1.2	Cel i założenia projektu	3
1.3	Idea systemu i urządzeń	4
1.4	Wstępny dobór komponentów	4
2	Opis teoretyczny protokołów CAN i BLE	5
2.1	BLE - Bluetooth Low Energy	5
2.2	CAN - Controller Area Network	6
2.3	OBD2 - On-Board Diagnostics, Second Generation	6
3	Opracowanie systemu	8
3.1	Rozwiązanie komunikacji bezprzewodowej BLE	8
3.2	Stacja OBD2	10
3.3	Stacja z wyświetlaczem	13
3.4	Prototypy obudów	15
4	Testowanie systemu	16
4.1	Komunikacja CAN	16
4.2	Połączenie BLE	17
4.3	Testowanie w samochodzie starszej generacji	18
4.4	Testowanie w samochodzie nowszej generacji	21
4.5	Testowanie spalania	22
5	Wnioski, perspektywy rozwoju systemu	25
5.1	Podsumowanie prac	25
5.2	Perspektywy rozwoju	25
	Bibliografia	27
	Spis rysunków	28
	Spis tabel	29

Rozdział 1

Wstęp, cel i założenia projektu

W rozdziale opisano główną ideę urządzeń, cel i założenia projektu. Przeprowadzono dobór oraz analizę komponentów wymaganych do uzyskania podstawowych funkcjonalności systemu.

1.1 Wprowadzenie w zagadnienie

Interfejs OBD2 umożliwia prostą i ogólną diagnostykę systemów samochodowych, bez konieczności bardziej złożonych operacji. Możliwości te są szeroko wykorzystywane przez skanery OBD2 dostępne na rynku głównie w celu odczytu kodu usterki lub resetowania niegroźnych błędów. Samochody wyprodukowane po 2001 z silnikami benzynowymi są obowiązkowo wyposażone w ten interfejs, natomiast dla silników Diesla, wymóg ten pojawił się dopiero w 2004 roku.

OBD2 umożliwia nie tylko diagnostykę, ale także wykonywanie testów oraz odczyt obecnych parametrów pracy samochodu. W projekcie wykorzystano właśnie te informacje, pomijając dane diagnostyczne, co jednak może bez problemu zostać rozwinięte w przyszłości.

1.2 Cel i założenia projektu

Celem projektu jest skonstruowanie systemu urządzeń działającego w samodzielnym ekosystemie. Mimo szerokiego wyboru skanerów OBD2 na rynku, często oprogramowanie do nich jest dodatkowe płatne lub mało czytelne.

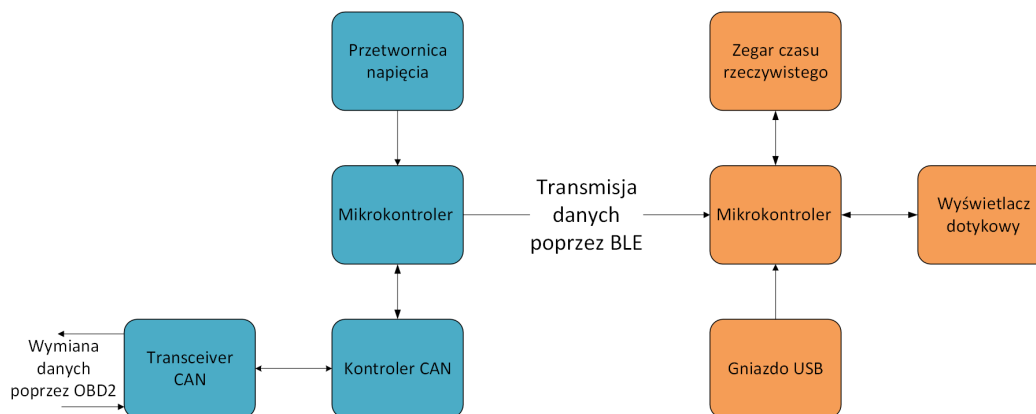
Główną motywacją projektu było skonstruowanie urządzenia minimalistycznego, umożliwiającego dodawanie własnych funkcjonalności, co często nie jest możliwe w gotowych rozwiązaniach zamknięto-programowych.

Projekt zakłada wykonanie dwóch stacji:

- Stacji OBD2 - odczytującej dane z magistrali CAN w samochodzie poprzez interfejs OBD2 i przekazującej je protokołem BLE do drugiego urządzenia.
- Stacji z wyświetlaczem - Prostego urządzenia z wyświetlaczem dotykowym, umożliwiającym podgląd najważniejszych informacji, a także resetowanie przeszłych danych jak w przypadku prostego licznika kilometrów w samochodzie.

1.3 Idea systemu i urządzeń

Poniżej przedstawiono idea działania systemu z rozdzieleniem na dwa wcześniej opisane urządzenia. Na niebiesko zaznaczono stację OBD2, a na pomarańczowo stację z wyświetlaczem.



Rysunek 1.1: Idea systemu

1.4 Wstępny dobór komponentów

Po zdefiniowaniu funkcjonalności urządzeń przystąpiono do wstępnego doboru komponentów.

Stacja OBD2

Jako serce układu - mikrokontroler wybrano układ firmy Espressif ESP32 C3. Procesor ten jest teoretycznie jednym z najsłabszych przedstawicieli rodziny ESP32 jednak wystarczającym, aby odczytać dane z magistrali CAN oraz przesłać je do dalszych obliczeń na drugą stację poprzez BLE.

Do komunikacji poprzez CAN wymagane są dwa układy. Moduł kontrolera oraz moduł transceivera. Jako kontroler wybrano szeroko stosowany w modułach deweloperskich - MCP2515. Aby ujednolicić zasilanie do 3.3 V i uniknąć stosowania kolejnej przetwornicy na 5 V wybrano transceiver SN65HVD230DR, nieco rzadziej wykorzystywany niż standardowy TJA1050, który wymaga zasilania 5 V.

Aby zasilić układ wybrano przetwornicę firmy Recom Power. Zasilanie udostępnione przez złącze diagnostyczne w samochodach wynosi 12 V. Sekcja logiczna urządzenia potrzebuje do stabilnego działania 3.3 V. Dopasowano zatem model R-78K3.3-1.0 umożliwiający ustabilizowane zasilanie 3.3 V do 1 A.

Stacja z wyświetlaczem

W przypadku stacji z wyświetlaczem zastosowano wydajniejszy mikrokontroler, ale także z rodziny ESP32, tym razem model S3, który także wspiera komunikację BLE. Zastosowanie mocniejszej jednostki obliczeniowej umożliwia w przyszłości rozwój urządzenia, a także obsługę większych wyświetlaczy.

Ze względu na stosunkowo małą ilość danych wybrano wyświetlacz 2.8 cala wraz z możliwością użycia kontrolera dotyku.

Za zegar czasu rzeczywistego, jako dodatkową funkcjonalność wybrano MCP79411, który integruje jednocześnie pamięć EEPROM z możliwością podtrzymania informacji o czasie i dacie na czas wyłączenia urządzenia poprzez baterię pastylkową.

Rozdział 2

Opis teoretyczny protokołów CAN i BLE

W rozdziale opisano protokoły komunikacyjne BLE(Bluetooth Low Energy) oraz CAN(Controller Area Network). Drugi z protokołów został wykorzystany do odczytu danych diagnostycznych z gniazda OBD2, natomiast drugi do przesyłu tych danych bezprzewodowo do stacji z wyświetlaczem.

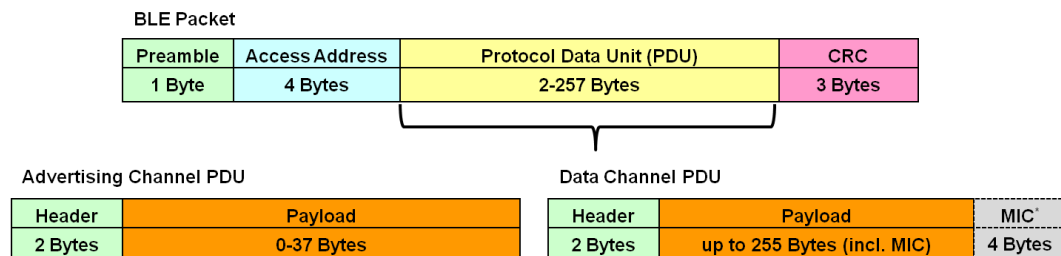
2.1 BLE - Bluetooth Low Energy

Bluetooth Low Energy (BLE) jest technologią bezprzewodową krótkiego zasięgu, zoptymalizowaną pod kątem niskiego poboru energii. Komunikacja BLE opiera się na dwóch podstawowych mechanizmach: advertising oraz GATT (Generic Attribute Profile).

Advertising jest mechanizmem rozgłaszania obecności urządzenia w sieci BLE. Urządzenie peryferyjne cyklicznie nadaje pakiety reklamowe, które zawierają m.in. nazwę urządzenia, identyfikatory usług (UUID) oraz informacje specyficzne dla producenta. W klasycznym BLE maksymalna długość danych reklamowych wynosi 31 bajtów, co ogranicza ilość informacji przesyłanych przed nawiązaniem połączenia. Klient (np. smartfon) skanuje eter i wykrywa urządzenia na podstawie pakietów advertising, po czym może zainicjować połączenie.

Po zestawieniu połączenia komunikacja odbywa się w modelu GATT, w którym serwer udostępnia usługi i charakterystyki, a klient może wykonywać operacje typu **read**, **write** oraz **notify**. Każda usługa i charakterystyka jest identyfikowana za pomocą UUID (16- lub 128-bitowego). W tym modelu możliwa jest transmisja rzeczywistych danych diagnostycznych, takich jak odczyt parametrów pojazdu z interfejsu OBD-II.

Ramka Advertising PDU składa się z 2-bajtowego nagłówka oraz maksymalnie 37-bajtowego pola danych, w którym mieszczą się nazwa urządzenia i identyfikatory usług. Po zestawieniu połączenia właściwe dane GATT przesyłane są w Data Channel PDU, zawierającym 2-bajtowy nagłówek, pole danych do 255 bajtów oraz 4-bajtowy MIC dla zapewnienia integralności transmisji.



Rysunek 2.1: Format ramek BLE[1]

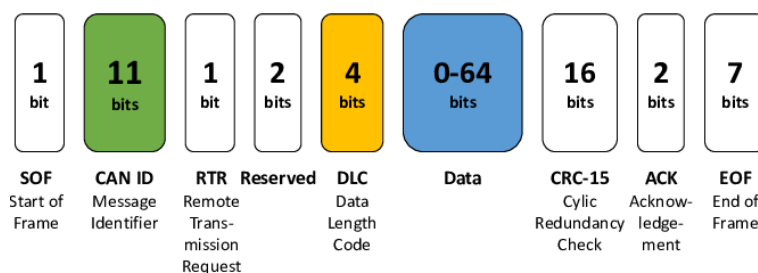
2.2 CAN - Controller Area Network

CAN (Controller Area Network) jest szeregową magistralą komunikacyjną stosowaną powszechnie w systemach elektronicznych pojazdów. Została opracowana przez firmę Bosch w celu zapewnienia niezawodnej i odpornej na zakłócenia wymiany danych pomiędzy sterownikami ECU.

Magistrala CAN wykorzystuje transmisję różnicową po liniach CAN_H oraz CAN_L, co zwiększa odporność na zakłócenia elektromagnetyczne. Komunikacja odbywa się bez centralnego sterownika nadrzędnego, a dostęp do magistrali realizowany jest metodą arbitrażu bitowego opartego na priorytecie identyfikatora ramki.

W systemach diagnostycznych OBD-II magistrala CAN, zgodna z normą ISO 15765-4, jest obecnie najczęściej stosowanym protokołem transmisyjnym. Zapewnia ona wysoką prędkość transmisji (do 1 Mb/s) oraz niezawodną komunikację pomiędzy interfejsem diagnostycznym a sterownikami pojazdu.

Ramka CAN składa się z kilku pól: identyfikatora, pola sterującego, danych (0–8 bajtów w klasycznej wersji), sumy kontrolnej CRC oraz bitów potwierdzenia i zakończenia ramki. Identyfikator ramki określa priorytet transmisji, a mechanizm arbitrażu bitowego zapewnia, że komunikaty o wyższym priorytecie mają pierwszeństwo dostępu do magistrali. Każda ramka zawiera także mechanizmy wykrywania błędów i potwierdzenia odbioru, co zapewnia niezawodną komunikację między sterownikami ECU.



Rysunek 2.2: Format ramek CAN[5]

Przesyłając dane głównie należy skupić się na poprawnym ustawieniu: CAN ID, DLC oraz oczywiście danych. Pozostałe bity ramki są zazwyczaj ustawiane przez wewnętrzne mechanizmy kontrolerów CAN.

2.3 OBD2 - On-Board Diagnostics, Second Generation

OBD-II (On-Board Diagnostics, Second Generation) jest ustandaryzowanym systemem diagnostyki pokładowej pojazdów, którego celem jest monitorowanie pracy silnika oraz układów odpowiedzialnych za emisję spalin. Standard ten definiuje jednolite złącze diagnostyczne, zestaw protokołów komunikacyjnych oraz formaty kodów błędów.

System OBD-II umożliwia odczyt parametrów bieżących pojazdu, takich jak prędkość obrotowa silnika, prędkość pojazdu, temperatura cieczy chłodzącej czy masowy przepływ powietrza. Dane te udostępniane są za pomocą identyfikatorów parametrów PID (Parameter ID).

OBD-II obsługuje kilka protokołów komunikacyjnych, w tym ISO 9141-2, ISO 14230 (KWP2000) oraz ISO 15765-4 (CAN). W nowoczesnych pojazdach standardem jest komunikacja poprzez magistralę CAN. Odczyt informacji diagnostycznych realizowany jest poprzez różne tryby pracy, m.in. tryb odczytu danych bieżących, tryb odczytu kodów usterek oraz tryb kasowania błędów. Odczytywać dane można w sumie z 10 różnych serwisów.

Mode Number	Mode Description
01	Current Data
02	Freeze Frame Data
03	Diagnostic Trouble Codes
04	Clear Trouble Code
05	Test Results/Oxygen Sensors
06	Test Results/Non-Continuous Testing
07	Show Pending Trouble Codes
08	Special Control Mode
09	Request Vehicle Information
0A	Request Permanent Trouble Codes

Rysunek 2.3: Serwisy dostępne w ramach interfejsu OBD2[2]

Aby uzyskać od sterownika ECU dane informację należy najpierw wysłać zapytanie. W ramce CAN należy zdefiniować:

- ID: 0x7DF,
- długość DLC: 8,
- Pierwszy bajt danych - długość danych: 2,
- Drugi bajt danych - tryb: 0-10 (w wypadku badania obecnych parametrów = 0x01)
- Trzeci bajt danych - PID (ID parametru): 0-200,
- Pozostałe bajty - padding: np. 0x55.

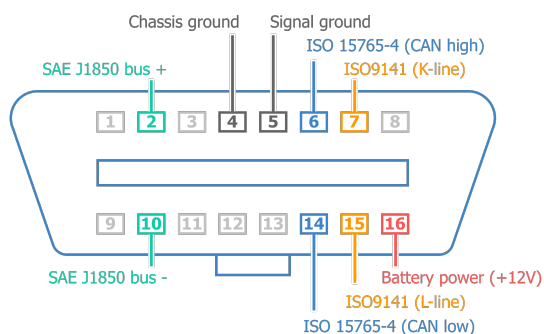
Następnie odpowiedź od sterownika otrzymujemy w formie ramki, z której dane są zawarte w bajtach 4-7 ramki CAN.

Identifier	#bytes	Mode	PID	A	B	C	D	Unused
(e.g. 7E8)	03	41	0D	32	AA	AA	AA	AA
CAN ID	CAN Data							

Rysunek 2.4: Ramka OBD2[6]

Aby podłączyć się do magistrali CAN z interfejsem OBD2 wymagane jest specjalne złącze. Piny potrzebne do komunikacji poprzez CAN to:

- 6 oraz 14 - linie transmisyjne
- 5 - masa odniesienia sygnału
- 16 - opcjonalne zasilanie układu



Rysunek 2.5: Wyprowadzenia gniazda OBD2[3]

Rozdział 3

Opracowanie systemu

W poniższym rozdziale opisano sposób implementacji komunikacji bezprzewodowej BLE. Wybrano dokładne komponenty i części, a następnie zaprojektowano płytki obwodów drukowanych obydwu urządzeń. Zaprezentowano docelowe wersje programów. Całość dopełniono prostymi obudowami wydrukowanymi w technologii druku 3D.

3.1 Rozwiązanie komunikacji bezprzewodowej BLE

W opracowanym urządzeniu komunikacja bezprzewodowa zrealizowana została w technologii Bluetooth Low Energy (BLE) w modelu klient-serwer z użyciem profilu GATT (Generic Attribute Profile). Mikrokontroler pełniący rolę serwera udostępniał usługę GATT zawierającą charakterystykę diagnostyczną, pozwalającą na odczyt danych związanych z interfejsem OBD-II.

Mechanizm advertising, realizowany w ramach profilu GAP (Generic Access Profile), wykorzystywany był wyłącznie do rozgłaszania obecności urządzenia oraz umożliwienia jego wykrycia przez klienta. W danych reklamowych serwer przysyłał nazwę urządzenia oraz informację o dostępnej usłudze. Po wykryciu urządzenia przez klienta następowało zestawienie połączenia i rozpoczęcie transmisji danych w modelu GATT.

Do identyfikacji serwisu oraz charakterystyki zastosowano niestandardowe, 16-bitowe identyfikatory UUID, zdefiniowane w kodzie projektu jako:

- SERVICE_UUID = "OBD2" – identyfikator serwisu diagnostycznego,
- CHARACTERISTIC_UUID = "CODE" – identyfikator charakterystyki do odczytu i zapisu danych.

Serwer BLE

Do funkcjonowania serwera wykorzystano szereg bibliotek NimBLE. NimBLE jest lekkim stosem Bluetooth Low Energy, który działa jako warstwa hosta BLE, współpracując z kontrolerem radiowym Bluetooth.

Większość funkcjonalności BLE udostępnione jest przez API, a implementując mechanizmy advertisingu oraz połączenia wzorowano się na przykładach dostarczanych przez firmę Espressif.

Zdefiniowano niestandardową usługę GATT o identyfikatorze UUID 0x0BD2, zawierającą dwie charakterystyki. Pierwsza z nich, o UUID 0xC0DE, umożliwia odczyt danych diagnostycznych przez klienta BLE. Druga charakterystyka, o UUID 0xDEAD, służy do zapisu danych przesyłanych z urządzenia klienckiego do serwera BLE, nie jest ona jednak wykorzystywana w systemie.

```

1 static const struct ble_gatt_svc_def gatt_svcs[] = {
2     {
3         .type = BLE_GATT_SVC_TYPE_PRIMARY,
4         .uuid = BLE_UUID16_DECLARE(0x0BD2),
5         .characteristics = (struct ble_gatt_chr_def[]){
6             {
7                 .uuid = BLE_UUID16_DECLARE(0xC0DE),
8                 .flags = BLE_GATT_CHR_F_READ,
9                 .access_cb = device_read
10            },
11            {
12                .uuid = BLE_UUID16_DECLARE(0xDEAD),
13                .flags = BLE_GATT_CHR_F_WRITE,
14                .access_cb = device_write
15            },
16            {0}
17        }
18    },
19    {0}
20 };

```

Listing 3.1: Definicja usługi i charakterystyk GATT

Funkcja 3.2 jest wywoływana automatycznie przez stos BLE w momencie zapisu danych do charakterystyki przez urządzenie klienckie. Odebrane dane są buforowane w strukturze `os_mbuf`, z której następnie mogą zostać przetworzone przez aplikację.

```

1 static int device_write(uint16_t conn_handle,
2                        uint16_t attr_handle,
3                        struct ble_gatt_access_ctxt *ctxt,
4                        void *arg)
5 {
6     printf("Data from the client: %.*s\n",
7           ctxt->om->om_len,
8           ctxt->om->om_data);
9     return 0;
10 }

```

Listing 3.2: Funkcja obsługi zapisu danych z klienta BLE

Klient BLE

Po Stronie klienta wykorzystano także stos nimBLE jednak w formie uproszczonego API, ponieważ program na stacji z wyświetlaczem został napisany we frameworku Arduino, ze względu na dodatkową obsługę wyświetlacza i bibliotek graficznych.

Urządzenie klienckie realizuje aktywne skanowanie w poszukiwaniu urządzeń reklamujących się w trybie BLE Advertising. Po wykryciu urządzenia o zadanej nazwie następuje zatrzymanie skanowania oraz inicjalizacja procesu połączenia.

Po zestawieniu połączenia klient BLE wyszukuje usługę o określonym identyfikatorze UUID, a następnie uzyskuje dostęp do wybranej charakterystyki, wykorzystywanej do odczytu danych diagnostycznych.

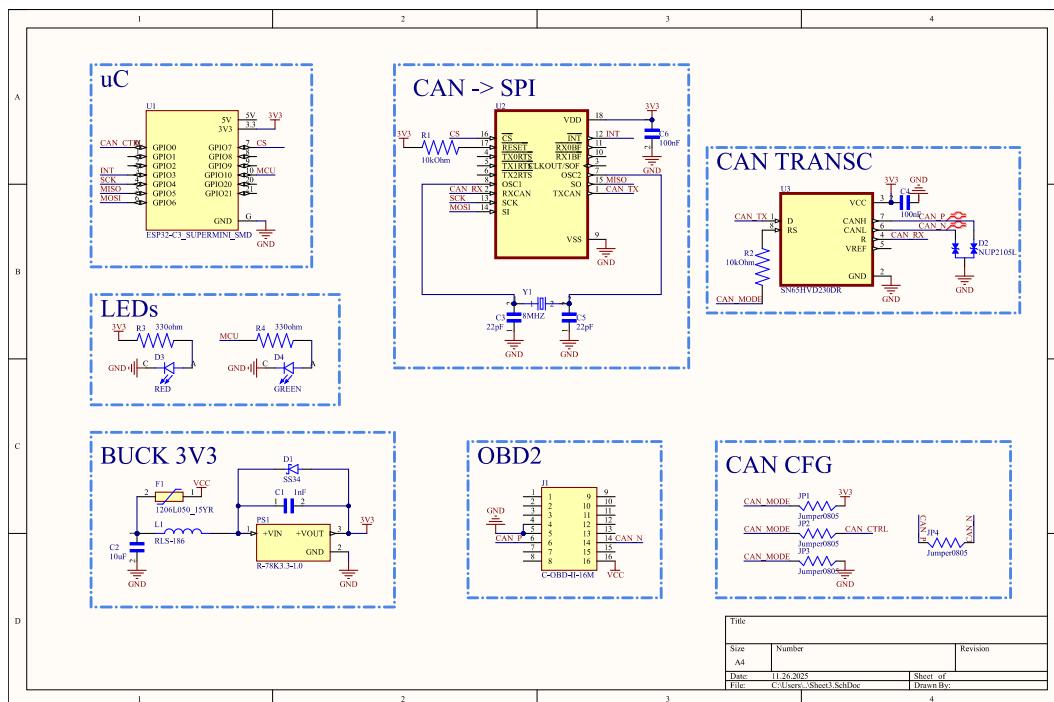
Obsługa zdarzeń BLE została zrealizowana w oparciu o mechanizm funkcji zwrotnych (callback), umożliwiających reagowanie na zmianę stanu połączenia oraz odbiór danych asynchronicznych w postaci notyfikacji.

Dane diagnostyczne odbierane są przez klienta BLE poprzez cykliczny odczyt wartości charakterystyki GATT, a następnie zapisywane do bufora aplikacyjnego w celu dalszego przetwarzania.

3.2 Stacja OBD2

Prace nad stacją OBD2 rozpoczęto od utworzenia schematu elektronicznego urządzenia. Oprócz wcześniej wybranych komponentów dołożono oczywiste elementy pasywne służące do zasilania lub wybrane konfiguracji poszczególnych układów scalonych. Dodatkowo dołożono dwie diody sygnalizujące zasilanie oraz pracę urządzenia.

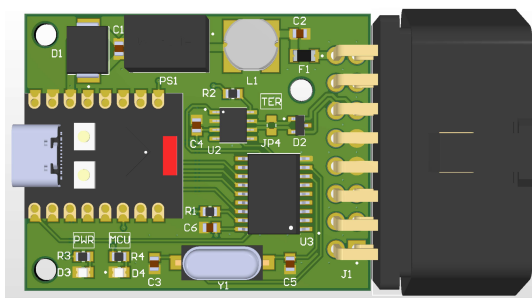
Kluczowym elementem było zdefiniowanie sygnałów CAN_H oraz CAN_L jako pary różnicowej, aby zapewnić później impedancję podczas projektowania płytki, czyli około 120 Ω . Dodatkowo zaprojektowano pady na miejsce których możliwe jest wlutowanie rezystora 120 Ω w przypadku testowania układu poza gniazdem OBD2, gdzie nie ma żadnej innej dodatkowej terminacji.



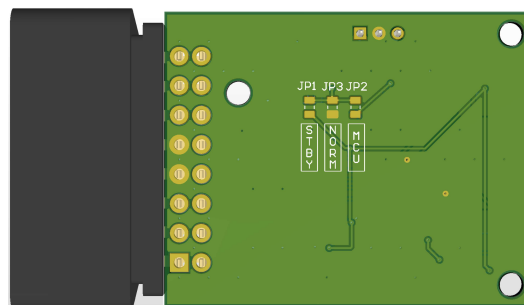
Rysunek 3.1: Schemat elektroniczny stacji OBD2

Dla lepszej diagnozy i rozwoju urządzenia, wszystkie komponenty umieszczono na górnej warstwie płytki. Mimo komponentów w dość dużych obudowach, aby umożliwić lutowanie ręczną płytką została zaprojektowana w nieco mniejszych wymiarach niż typowe skanery OBD2.

Zastosowano mikroprocesor w wersji deweloperskiej Supermini, aby w razie problemów, awarii umożliwić jego łatwą i szybką wymianę.



(a) Widok płytki z góry



(b) Widok płytki z dołu

Rysunek 3.2: Wizualizacji płytki stacji OBD2

Do odczytu danych po magistrali CAN zastosowano bibliotekę dla układu MCP2515. Oprócz standardowej jej inicjalizacji, wymagana była dość duża inna ilość operacji, aby dostosować odczyt pod interfejs OBD2.

Ze względu na inne dane przesyłane na magistrali CAN w systemie samochodowym, wymagane było skonfigurowanie filtrów układu, aby odbierać tylko wiadomości od sterownika silnika ECU.

```

1 // MASK0 - filters RXF0, RXF1 ext flag false
2 MCP2515_setFilterMask(MASK0, false, 0x7F0); // maska 11-bitowa
   standard ID
3 MCP2515_setFilter(RXF0, false, 0x7DF); // filter 0 functional
   request
4 MCP2515_setFilter(RXF1, false, 0x7E1); // filter 1 ECU response
5 // MASK1 - filters RXF2-RXF5
6 MCP2515_setFilterMask(MASK1, false, 0x7F0);
7 MCP2515_setFilter(RXF2, false, 0x7DF);
8 MCP2515_setFilter(RXF3, false, 0x7E1);
9 MCP2515_setFilter(RXF4, false, 0x7DF);
10 MCP2515_setFilter(RXF5, false, 0x7E1);

```

Listing 3.3: Konfiguracja filtrów MCP2515

Do odpytywania sterownika o poszczególne PIDy oraz odczyt odpowiedzi bazowano na własnych funkcjach OBD_write oraz OBD_read.

```

1 void OBD_write(uint8_t mode, uint8_t pid)
2 {
3     CAN_FRAME_t frame[1];
4
5     frame[0]->can_id = 0x7DF; // broadcast OBD2
6     frame[0]->can_dlc = 8; // dlugosc ramki OBD2 zawsze 8 bajtow
7     frame[0]->can_dlc = 8; // dlugosc ramki OBD2 zawsze 8 bajtow
8     frame[0]->data[1] = mode; // tryb diagnostyczny
9     frame[0]->data[2] = pid; // identyfikator parametru (PID)
10    frame[0]->data[3] = 0x55; // bajty wypelniajace (padding)
11    frame[0]->data[4] = 0x55;
12    frame[0]->data[5] = 0x55;
13    frame[0]->data[6] = 0x55;
14    frame[0]->data[7] = 0x55;
15
16    if (MCP2515_sendMessageAfterCtrlCheck(frame[0]) != ERROR_OK)
17        ESP_LOGE(TAG, "Wyslanie ramki OBD nie powiodlo sie");
18 }

```

Listing 3.4: Wysyłanie ramki OBD2

```

1 bool OBD_read(uint8_t *buf)
2 {
3     CAN_FRAME_t frame[1];
4
5     // Odczyt tylko gdy pin przerwania jest niski
6     if(!gpio_get_level(PIN_NUM_INTERRUPT))
7     {
8         uint8_t irq = MCP2515_getInterrupts();
9
10        // Odczyt z odpowiedniego bufora RX
11        if (irq & CANINTF_RX0IF)
12        {
13            if (MCP2515_readMessage(RXB0, frame[0]) != ERROR_OK)
14                return false;
15        }
16        else if (irq & CANINTF_RX1IF)
17        {
18            if (MCP2515_readMessage(RXB1, frame[0]) != ERROR_OK)
19                return false;
20        }
21        // Sprawdzenie, czy ramka pochodzi od ECU (0x7E8)
22        if (frame[0]->can_id != 0x7E8)
23            return false;
24        // Skopiowanie danych ramki do bufora
25        for (int i = 0; i < frame[0]->can_dlc; i++) {
26            buf[i] = frame[0]->data[i];
27        }
28        return true;
29    }
30    return false;
31 }

```

Listing 3.5: Odczyt ramki OBD2

Zgodnie z dokumentacją OBD2 z odpowiednich PIDów cyklicznie uzyskiwano dane o: prędkości, obrotach silnika oraz przepływie masy powietrza. Starsze samochody nie udostępniają przepływu MAF, w tym przypadku należało skorzystać z dodatkowej formuły opartej na temperaturze, ciśnieniu oraz obrotach silnika. Ze względu na dodatkowe stałe jak masa molowa, stała gazowa oraz parametry silnika obliczenia te zmniejszały dokładność pomiaru spalania paliwa.

Ostatecznie dane były pakowane do bufora BLE, w celu wysyłki ich do stacji z wyświetlaczem.

```

1 ble_buf[0] = velocity;           // predkosc pojazdu
2 ble_buf[1] = (RPM >> 8) & 0xFF; // gorny bajt RPM
3 ble_buf[2] = RPM & 0xFF;         // dolny bajt RPM
4 ble_buf[3] = (MAF >> 8) & 0xFF;  // gorny bajt MAF
5 ble_buf[4] = MAF & 0xFF;         // dolny bajt MAF

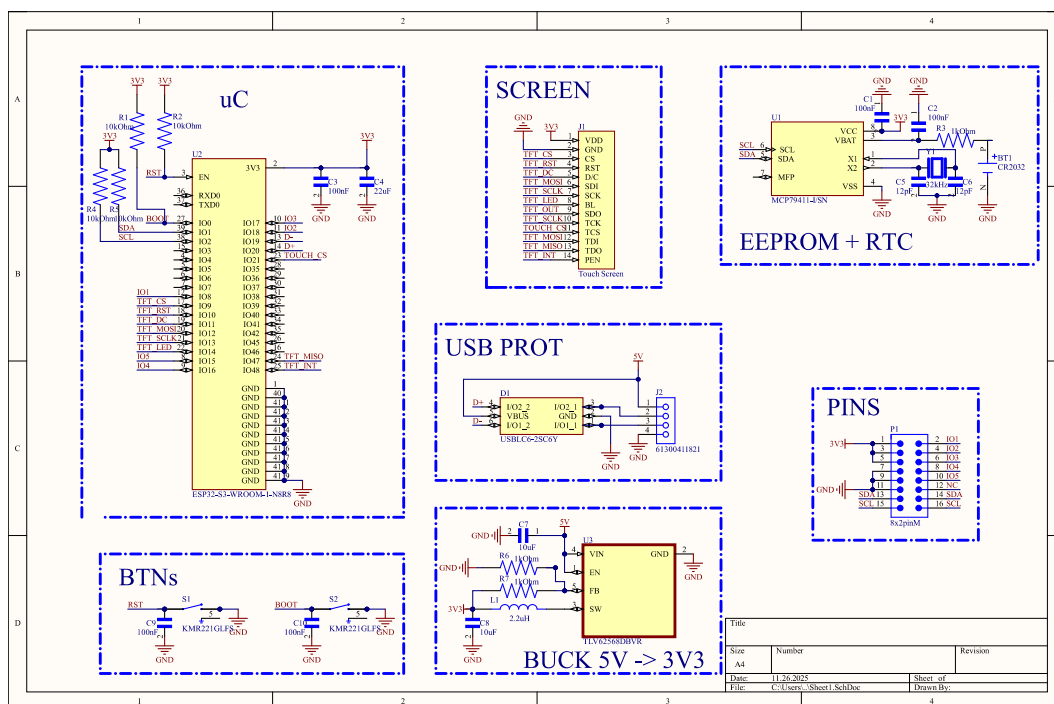
```

Listing 3.6: Przygotowanie danych do wysyłki przez BLE

3.3 Stacja z wyświetlaczem

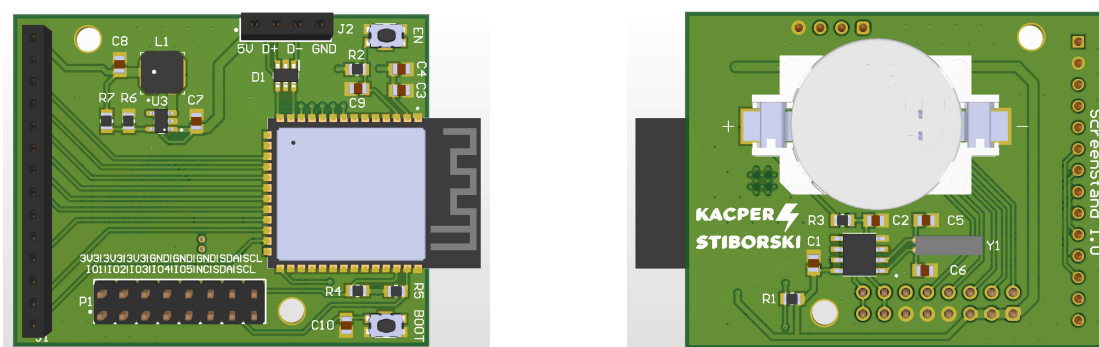
Stacja z wyświetlaczem zaprojektowano jako dodatkowe urządzenie, możliwe także do zastosowania w innych projektach. Oprócz wybranych wcześniej komponentów zastosowano także układ z diodami zabezpieczający gniazdo USB i możliwe wyładowania elektrostatyczne spowodowane przez użytkownika. Napięcie z USB przechodzi przez przetwornicę w celu uzyskania 3.3 V do zasilania mikroprocesora i innych modułów, w tym wyświetlacza.

Dodatkowo wyprowadzono kilka pinów zasilających i GPIO, aby umożliwić w przyszłości dołączenie dodatkowych modułów.



Rysunek 3.3: Schemat elektroniczny stacji z wyświetlaczem

Projektując płytkę ze względu na wymiary wyświetlaczy dotykowych nie skupiono się aż tak bardzo na jej minimalizacji. Zastosowano mikrokontroler w wersji WROOM co wymagało użycia większej ilości komponentów pasywnych, ale udostępniło większą liczbę jego wyprowadzeń oraz wysokość komponentu.



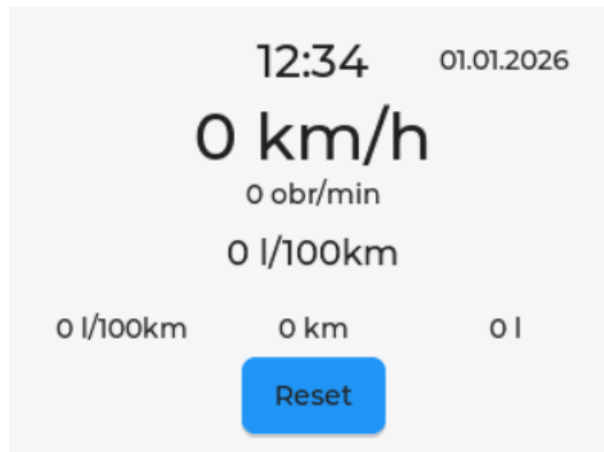
(a) Widok płytki z góry

(b) Widok płytki z dołu

Rysunek 3.4: Wizualizacji płytki stacji z wyświetlaczem

Następnie w programie SquareLineStudio zaprojektowano menu na wyświetlacz. Aplikacja pozwala wygenerować kod źródłowy, który po skonfigurowaniu bibliotek sterownika wyświetlacza oraz biblioteki LVGL (Light and Versatile Graphics Library) można zaimplementować w swoim projekcie.

Zdecydowano się na wyświetlanie danych o obecnej godzinie i dacie. Poniżej wyświetlana jest obecna prędkość i obroty silnika, a także spalanie chwilowe. Na samym dole wyświetlacza można obserwować informacje o spalaniu średnim, przejechanej drodze oraz ilości zużytego paliwa. Dane te można resetować za pomocą przycisku Reset.



Rysunek 3.5: Menu stacji z wyświetlaczem

Na podstawie danych pozyskanych z bufora BLE, cyklicznie wyliczono wspomniane wcześniej parametry, aby następnie wyświetlać je na ekranie.

```

1 void update_parameters()
2 {
3     // Aktualizacja wartosci z bufora BLE
4     speed = ble_buf[0];
5     RPM   = (ble_buf[1] << 8) | ble_buf[2];
6     MAF   = (ble_buf[3] << 8) | ble_buf[4];
7     Serial.printf("Speed: %d, RPM: %d, MAF: %d dt: %f\n", speed, RPM, MAF,
8         dt_s);
9
10    // Obliczanie parametrow
11    // Spalanie chwilowe (litry na godzinie)
12    float fuel_lph = (MAF * 3600.0) / (14.7 * 745.0); // AFR * gestosc
13    paliwa
14    if (speed > 1)
15        fuel_consumption = (fuel_lph / speed) * 100; // litry / 100 km
16    else
17        fuel_consumption = 0;
18
19    // Pokonana odleglosc
20    distance = distance + speed * (dt_s / 3600.0);
21
22    // Zuzyte paliwo
23    fuel = fuel + fuel_lph * (dt_s / 3600.0);
24
25    // Srednie spalanie
26    if (distance > 0.01)
27        fuel_consumption_avg = (fuel / distance) * 100;
28
29    // Debug

```

```

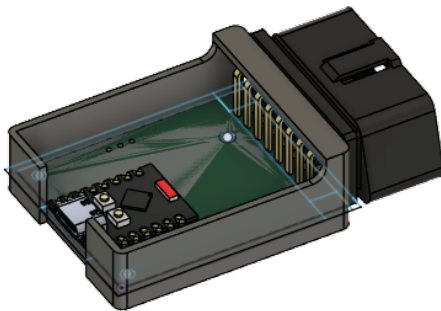
28 Serial.printf("LPH: %f, L100KM: %f, L100KM_AVG: %f, DIST: %f, FUEL: %f\n",
29             fuel_lph, fuel_consumption, fuel_consumption_avg,
30             distance, fuel);
31 // Reset wartosci sredniego spalania, odleglosci i zuzycia paliwa
32 if (reset == true)
33 {
34     fuel_consumption_avg = 0;
35     distance = 0;
36     fuel = 0;
37     reset = false;
38 }
39 }

```

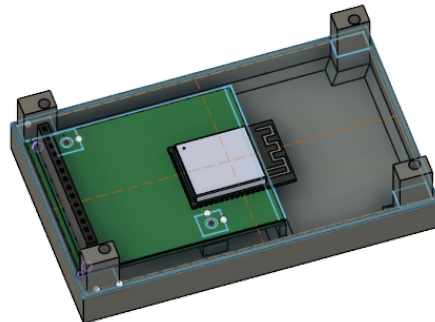
Listing 3.7: Aktualizacja parametrów z bufora BLE

3.4 Prototypy obudów

Modele płytek wyeksportowano do programu Fusion360, gdzie zaprojektowano proste prototypy obudów, a właściwie podstawek, aby mniej narazić komponenty elektroniczne na wyładowania elektrostatyczne.



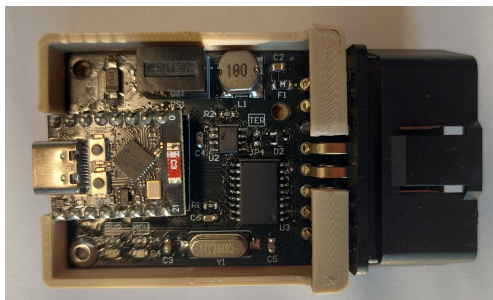
(a) Obudowa stacji OBD2



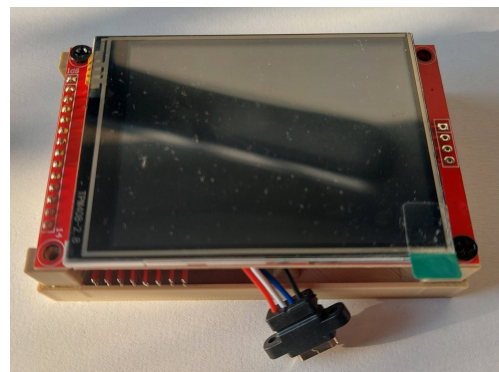
(b) Obudowa stacji z wyświetlaczem

Rysunek 3.6: Modele obudów

Następnie wydrukowano modele w technologii druku 3D z filamentu PLA i osadzono w nich płytki z użyciem śrub M3.



(a) Stacja OBD2



(b) Stacja z wyświetlaczem

Rysunek 3.7: Urządzenia wraz z obudowami

Rozdział 4

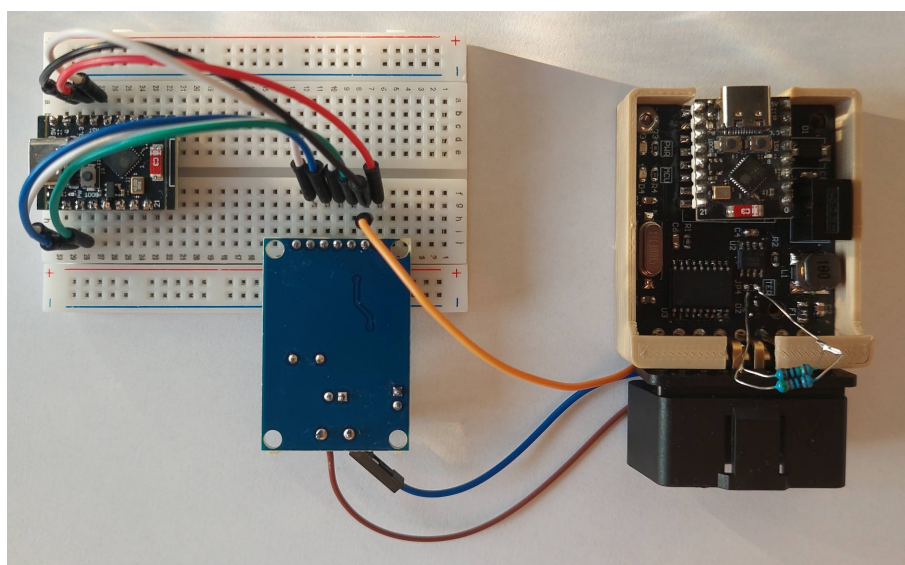
Testowanie systemu

W kolejnym rozdziale przetestowano urządzenia z osobna, a następnie w docelowym środowisku, czyli samochodzie. Przeanalizowano różne scenariusze użytkowania i wykonano ewentualne poprawki.

4.1 Komunikacja CAN

Kluczową funkcjonalnością projektu jest komunikacji poprzez interfejs OBD2. Aby ułatwić pracę nad komunikacją poprzez interfejs upewniono się najpierw co do poprawnego działania komunikacji poprzez magistralę CAN.

Na płytce stykowej zbudowano prosty nadajnik CAN, także na bazie modułu MCP2515 z transceiverem oraz mikrokontrolera w tej samej wersji deweloperskiej.



Rysunek 4.1: Testowanie komunikacji CAN

Zaimplementowano proste funkcje do wysyłania i odczytu danych na magistrali CAN. Przesyłano prosty licznik, inkrementowany o 1, po każdej wysyłce.

```
1 void CAN_write(void){
2     can_frame_tx[0]->data[0] = (counter >> 8) & 0xFF; // MSB
3     can_frame_tx[0]->data[1] = counter & 0xFF; // LSB
4     if(MCP2515_sendMessageAfterCtrlCheck(can_frame_tx[0]) != ERROR_OK) {
5         ESP_LOGE(TAG, "Could not send message.");
6     }
7     else {
8         ESP_LOGI(TAG, "Sent CAN value: %d", counter);
9     }
10    counter++;
11    if(counter > 65535) counter = 1;
12 }
```

Listing 4.1: Wysyłanie ramki CAN

```
1 void CAN_read(void){
2     if ((MCP2515_readMessage(RXB0, can_frame_rx[0]) == ERROR_OK) ||
3         (MCP2515_readMessage(RXB1, can_frame_rx[0]) == ERROR_OK)) {
4         printf("CAN ID: 0x%08lX\n", can_frame_rx[0]->can_id);
5         printf("DLC: %d\n", can_frame_rx[0]->can_dlc);
6         printf("Data: ");
7         for (int i = 0; i < can_frame_rx[0]->can_dlc; i++) {
8             printf("%02X", can_frame_rx[0]->data[i]);
9         }
10        printf("\n");
11    }
12    else {
13        printf("No message\n");
14    }
15 }
```

Listing 4.2: Odczyt ramki CAN

Po pomyślnej komunikacji z prostym nadajnikiem na płycie stykowej, urządzenie umieszczono w złączu OBD2 w rzeczywistym samochodzie. Uzyskano różne ramki, które nie były możliwe do zdekodowania. Prawdopodobnie były to dane z czujników poduszek powietrznych lub systemu ABS itp. Potwierdziło to jednak poprawny odczyt danych w magistrali CAN przez stację OBD2.

```
1 CAN ID: 0x00000360 DLC: 8 Data: 00 00 00 3E 54 00 00 00
2 CAN ID: 0x000004B0 DLC: 8 Data: 27 10 27 10 27 10 27 10
3 CAN ID: 0x00000201 DLC: 8 Data: 00 00 7D 00 27 10 00 7D
```

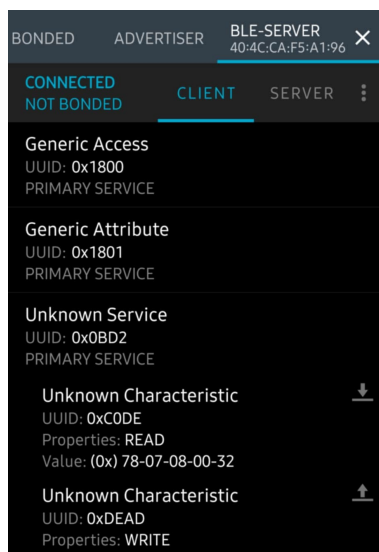
Listing 4.3: Odczyt ramki CAN

4.2 Połączenie BLE

Po prawidłowym odczycie danych przez CAN, przystąpiono do przetestowania komunikacji bezprzewodowej poprzez BLE. W tym celu skorzystano z aplikacji nRF connect. Ustawiono testowe wartości parametrów:

- Prędkość: 120,
- Obroty silnika: 1800,
- Przepływ masy powietrza: 50.

Następnie połączono telefon do serwera (stacji OBD2) i odczytano testowe dane.



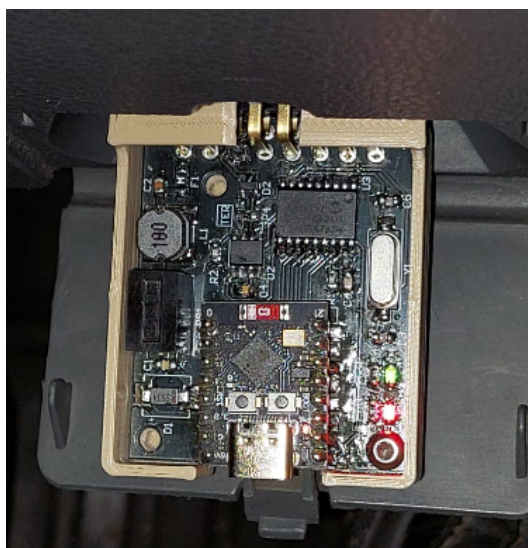
Rysunek 4.2: Testowanie komunikacji BLE

Z UUID "OBD2"z charakterystyki do odczytu "CODE"odczytano dane, które po zamianie z systemu szesnastkowego na dziesiętny odpowiadały testowym wartościom. Widoczny jest także rejestr do zapisu "DEAD".

4.3 Testowanie w samochodzie starszej generacji

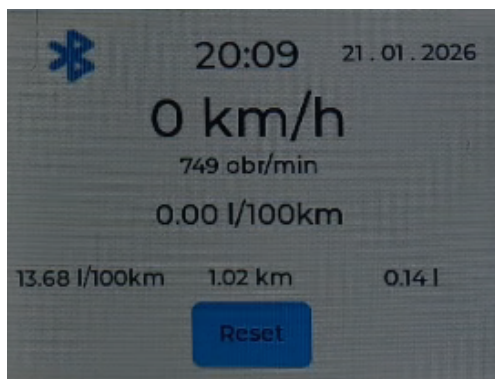
Urządzenia zostały przetestowane najpierw w samochodzie starszej generacji - Ford Fiesta (2003). Jako auto z silnikiem benzynowym, mimo dość dawnej daty produkcji, model ten posiada złącze OBD2 z komunikacją poprzez CAN.

Podłączono stację OBD2 do złącza diagnostycznego i po 5 sekundach zauważono mruganie zielonej diody. W kodzie jest to zaimplementowane jako pomyślne uzyskanie komunikacji po magistrali CAN z interfejsem OBD2.



Rysunek 4.3: Sygnalizacja połączenia z interfejsem OBD2

Następnie uruchomiono stację z wyświetlaczem, od razu pojawiła się ikonka BLE na wyświetlaczu, która oznacza zestawienie połączenia BLE ze stacją OBD2. Po wyjęciu stacji OBD2 ze złącza, ikonka BLE znika.



(a) Aktywne połączenie BLE



(b) Utrata połączenia BLE

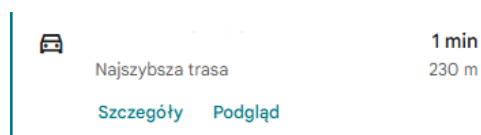
Rysunek 4.4: Sygnalizacja połączenia BLE

Przetestowano urządzenie na trzech różnych trasach. Ze względu na starszą generację samochodu, możliwe było tylko właściwie przetestowanie obliczeń przebytej drogi. Oczywiście możliwy jest też odczyt prędkości i obrotów silnika, jednak ze względu na analogową deskę rozdzielczą, oraz okres próbkowania 1 sekundy ciężko zestawić na jednym zdjęciu porównanie podczas jazdy samochodem.

Porównano obliczone przez zestaw urządzeń, komputer pokładowy samochodu oraz Mapy Google [4] przebyte drogi. Uzyskano bardzo dokładne wartości. Ostatni pomiar odbył się bez zresetowania wcześniejszego pomiaru, dlatego auto wskazuje 1.5 km, gdy tak naprawdę przebyto drogę 1 km.

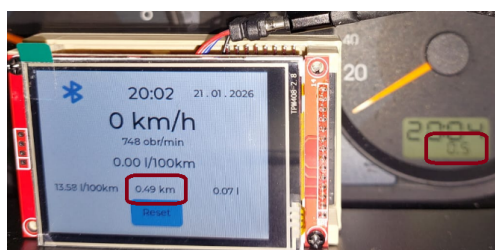


(a) Pomiar przebytej drogi

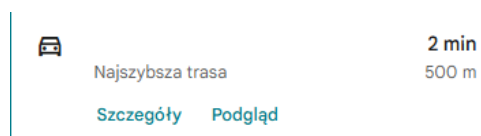


(b) Droga wyznaczona przez Google Maps

Rysunek 4.5: Porównanie przebytej drogi



(a) Pomiar przebytej drogi

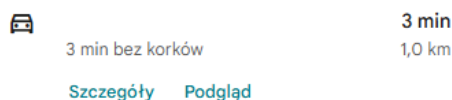


(b) Droga wyznaczona przez Google Maps

Rysunek 4.6: Porównanie przebytej drogi



(a) Pomiar przebytej drogi



(b) Droga wyznaczona przez Google Maps

Rysunek 4.7: Porównanie przebytej drogi

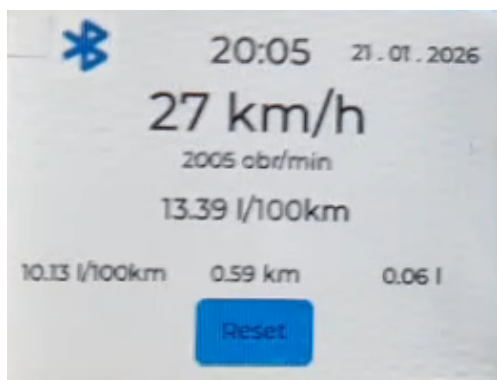
W przypadku samochodu starszej generacji nie uzyskano dostępu do rejestru MAF, w związku z czym obliczano spalanie paliwa ze wzoru:

$$MAF = \frac{RPM \cdot MAP}{IAT} \cdot \frac{VE}{100} \cdot \frac{ED \cdot MM}{120 \cdot R}$$

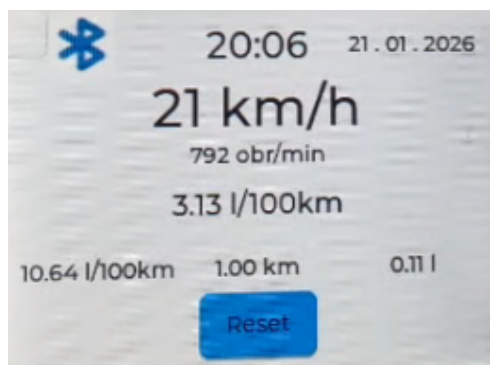
gdzie:

- RPM – prędkość obrotowa silnika [obr/min]
- MAP – ciśnienie bezwzględne w kolektorze dolotowym [kPa]
- IAT – temperatura powietrza dolotowego [K]
- VE – sprawność objętościowa silnika [%]
- ED – pojemność skokowa silnika [L]
- MM – masa molowa powietrza (28,97 g/mol)
- R – uniwersalna stała gazowa (8,314 J/(mol · K))

Niestety w tym przypadku obliczenia nie są tak dokładne, choćby ze względu na konieczność podania dodatkowo wydajności oraz pojemności skokowej silnika. O ile pojemność jest stała dla danego samochodu, tak wydajność zależy od stylu jazdy. Wartości te zostały w programie uśrednione.



(a) Przyspieszanie samochodu



(b) Niższa moc przepustnicy

Rysunek 4.8: Porównanie spalania samochodu przy różnym ustawieniu przepustnicy

Zauważyć można jednak pewną poprawność algorytmu. Przy przyspieszaniu, większej ilości obrotów silnika spalanie chwilowe rośnie, natomiast puszczając gaz, zmniejszając także liczbę obrotów spalanie chwilowe maleje.

4.4 Testowanie w samochodzie nowszej generacji

Następnie system przetestowano w nowszym samochodzie - Toyota Aygo (2023). Umożliwiło to porównanie danych z interfejsem komputera pokładowego a nawet z aplikacją MyToyota[7].

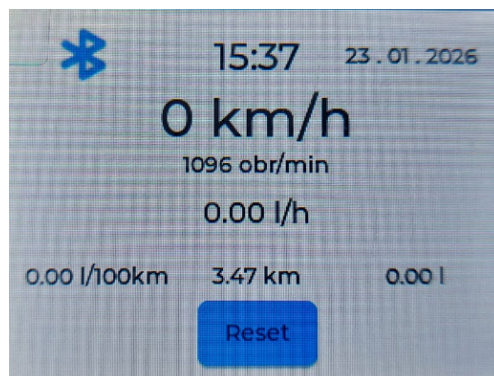
Bardzo dużym zaskoczeniem okazał się brak możliwości odczytu danych potrzebnych do wyliczenia spalania, nawet w okrojonej wersji jak w przypadku Forda Fiesty, starszego o 20 lat.

```
F (5479) main: Found PID!
06 41 00 BE 1E B8 13 00
I (5479) main: PID 0x01 supported
I (5479) main: PID 0x03 supported
I (5479) main: PID 0x04 supported
I (5489) main: PID 0x05 supported
I (5489) main: PID 0x06 supported
I (5489) main: PID 0x07 supported
I (5499) main: PID 0x0C supported
I (5499) main: PID 0x0D supported
I (5509) main: PID 0x0E supported
I (5509) main: PID 0x0F supported
I (5509) main: PID 0x11 supported
I (5519) main: PID 0x13 supported
I (5519) main: PID 0x14 supported
I (5519) main: PID 0x15 supported
I (5529) main: PID 0x1C supported
I (5529) main: PID 0x1F supported
I (5539) main: PID 0x20 supported
I (5539) main: OBD II connected
F (5549) main: Found PID!
06 41 00 BE 1E B8 13 00
W (5549) main: MAF PID 0x10 NOT supported
```

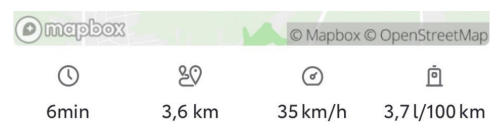
Rysunek 4.9: PIDy obsługiwane przez Toyota Aygo (2023)

Nie jest w tym wypadku dostępny rejestr 0x0F (temperatura powietrza dolotowego) co uniemożliwia wyliczenie informacji o spalaniu. Jest to dość nietypowy przypadek, lecz coraz powszechniejszy w niektórych samochodach, gdzie producenci celowo wycinają niektóre dane ze złącza diagnostycznego, pozostawiając tylko te obowiązkowe.

W przypadku tego samochodu możliwa była jedynie walidacja przejechanego dystansu, co porównano z aplikacją Toyoty.



(a) Pomiar przebytej drogi



(b) Droga wyznaczona przez aplikację Toyoty

Rysunek 4.10: Porównanie przebytej drogi

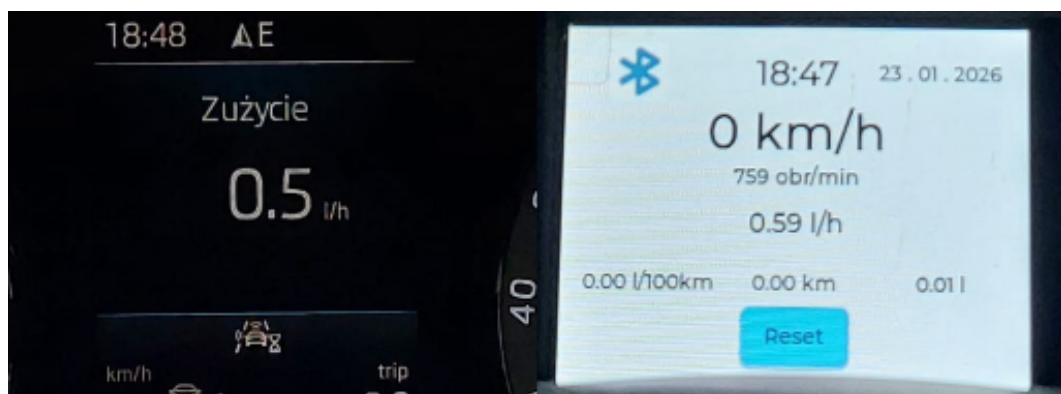
4.5 Testowanie spalania

Ostatecznie urządzenia przetestowano w samochodzie Skoda Kodiaq (2020). W tym przypadku producent udostępnia nawet rejestr Ox10, czyli właściwy przepływ powietrza w silniku co pozwala obliczyć dokładniej spalanie paliwa.

```
F (5479) main: Found PID!  
06 41 00 BE 1E B8 13 00  
I (5479) main: PID 0x01 supported  
I (5479) main: PID 0x03 supported  
I (5479) main: PID 0x04 supported  
I (5489) main: PID 0x05 supported  
I (5489) main: PID 0x06 supported  
I (5489) main: PID 0x07 supported  
I (5499) main: PID 0x0C supported  
I (5499) main: PID 0x0D supported  
I (5509) main: PID 0x0E supported  
I (5509) main: PID 0x0F supported  
I (5509) main: PID 0x11 supported  
I (5519) main: PID 0x13 supported  
I (5519) main: PID 0x14 supported  
I (5519) main: PID 0x15 supported  
I (5529) main: PID 0x1C supported  
I (5529) main: PID 0x1F supported  
I (5539) main: PID 0x20 supported  
I (5539) main: OBD II connected  
F (5549) main: Found PID!  
06 41 00 BE 1E B8 13 00  
W (5549) main: MAF PID 0x10 NOT supported
```

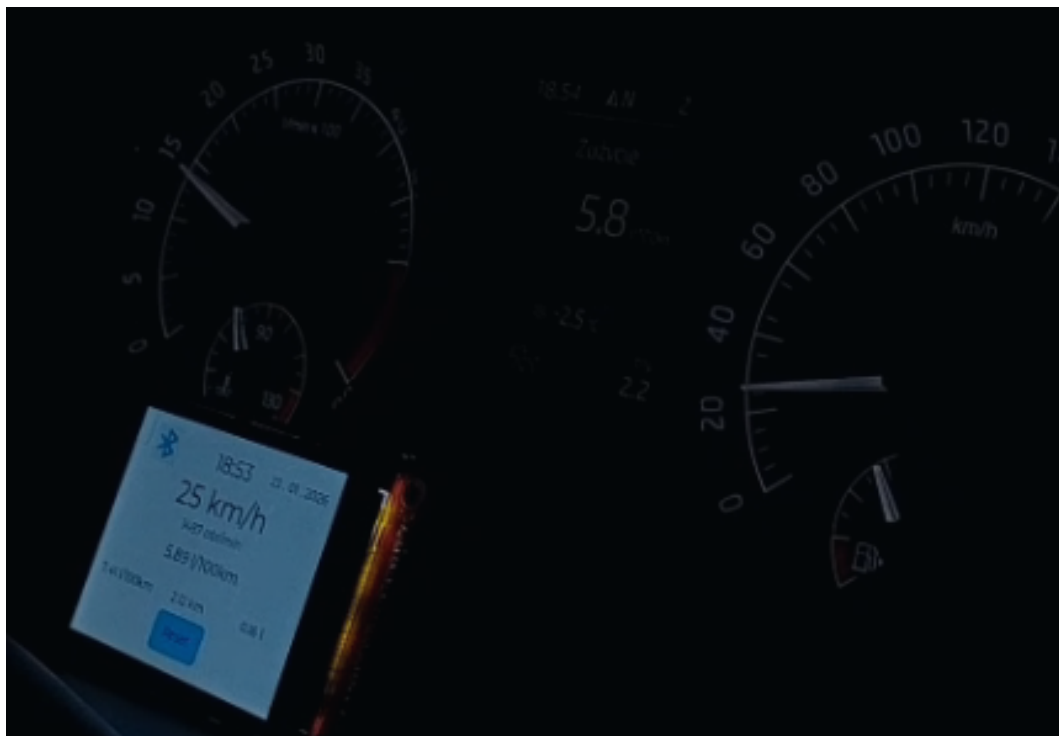
Rysunek 4.11: PIDy obsługiwane przez Skoda Kodiaq (2020)

Na postoju spalanie na stacji z wyświetlaczem było wyświetlane tak samo jak w samochodach na komputerach pokładowych, czyli w jednostce l/h. Obliczanie spalania w jednostce l/100km nie ma sensu fizyczne przy prędkości = 0.

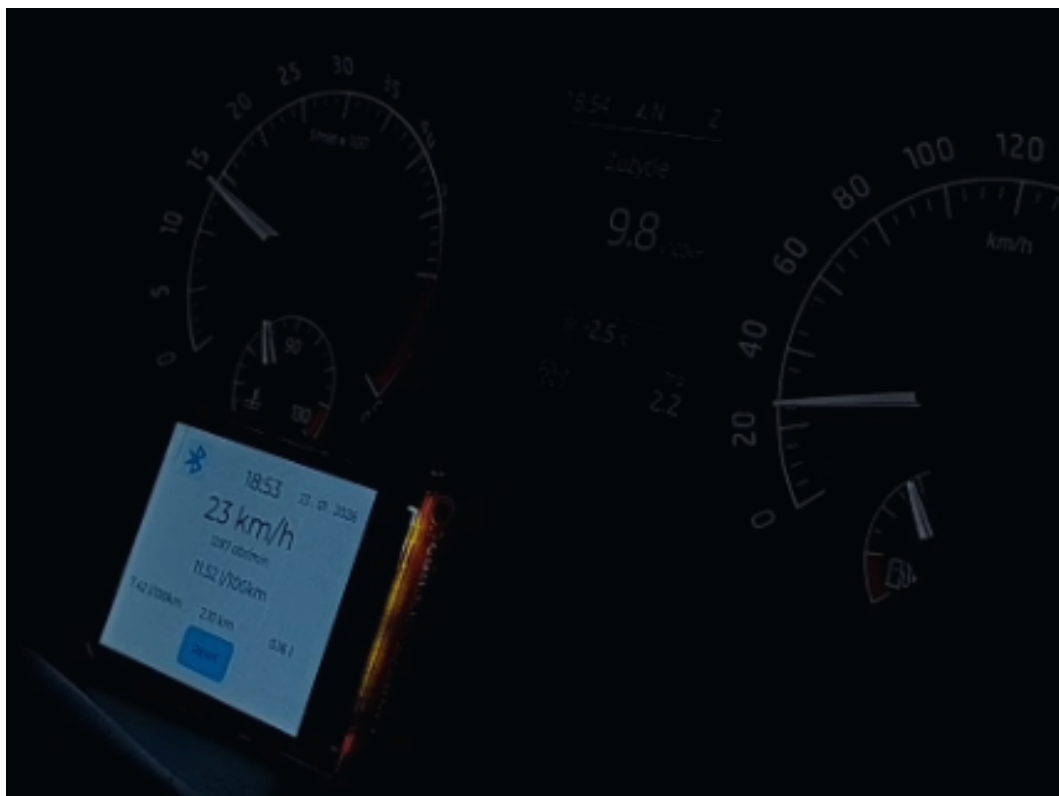


Rysunek 4.12: Spalanie na podstoju (l/h)

Następnie przetestowano pomiary podczas jazdy, w różnych momentach jazdy, dane pokrywały się z tymi wyświetlanymi na desce rozdzielczej.



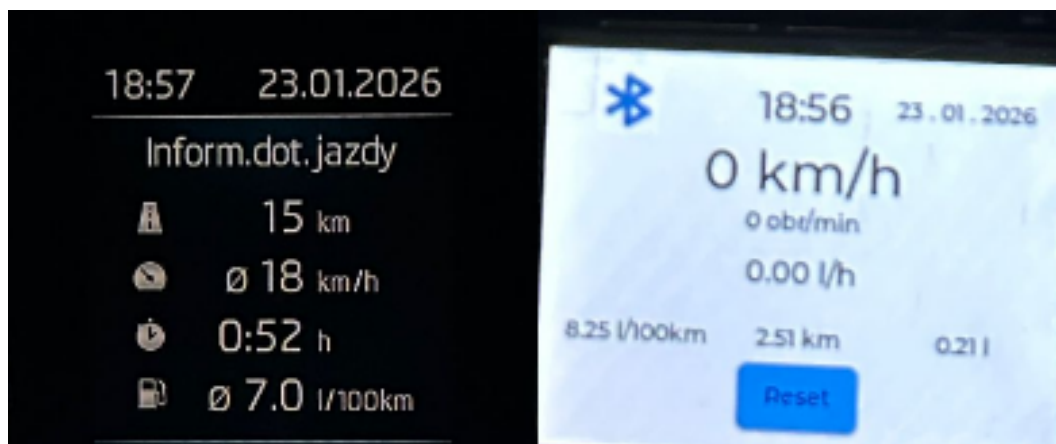
Rysunek 4.13: Spalanie podczas jazdy(l/100km)



Rysunek 4.14: Spalanie podczas jazdy(l/100km)

Ostatecznie porównano podsumowanie trasy wyświetlane przez samochód oraz stację z wyświetlaczem. Niestety ze względu na kilkukrotne uruchamianie i wyłączanie silnika w krótkim okresie czasu samochód nie rozdzielał tych tras.

Przewagą systemu zbudowanego podczas prac nad projektem jest możliwość zresetowania trasy w każdej chwili przyciskiem reset w takim przypadku.



Rysunek 4.15: Spalanie średnie na trasie (l/100km)

Rozdział 5

Wnioski, perspektywy rozwoju systemu

W ostatnim rozdziale opisano wnioski z wykonanych prac. Zaproponowano dalsze możliwości wykorzystania, a także rozwoju urządzenia.

5.1 Podsumowanie prac

W ramach projektu zaprojektowano, zbudowano oraz zaprojektowano system dwóch urządzeń pozwalający na odczyt danych z interfejsu diagnostycznego OBD2.

Zaprojektowano płytki obwodów drukowanych, napisano programy pod mikrokontrolery zarówno we frameworku Arduino, jak i bardziej niskopoziomowo w esp-idf. Zaimplementowano kompletną komunikację poprzez BLE oraz magistralę CAN, z wykorzystaniem interfejsu OBD2. Zaprojektowano i wkomponowano w program obsługę wyświetlacza dotykowego. Ostatecznie obudowano urządzenia wydrukami 3D.

System osiągnął zadowalającą dokładność oraz pozwala na wykorzystanie w autach starszych generacji, gdzie nie jest możliwy podgląd danych np. o spalaniu, ze względu na brak komputera pokładowego. Bardzo prostą metodą zwiększania dokładności może być zwiększenie częstotliwości próbkowania i przesyłania danych, co jest możliwe, ale wymaga odpowiednich badań.

Uzyskany system odróżnia od gotowych tego typu modułów, przede wszystkim niezależność od np. telefonu komórkowego. Pracuje on we własnym ekosystemie i nie ma potrzeby np. instalowania dodatkowych aplikacji. System pozwala także na dowolne modyfikacje w przyszłości. Stacja OBD2 jest dodatkowo minimalnie mniejsza od większości gotowych tego typu rozwiązań, mimo zastosowania dość sporych elementów na płycie PCB, ze względu na ręczne lutowanie.

Także w odróżnieniu od gotowych rozwiązań komunikacja jest zestawiona poprzez BLE, a nie Bluetooth Classic. Ze względu na wykorzystanie mikrokontrolerów z serii ESP32 jest możliwe zastosowanie nawet innych protokołów jak ESP-NOW lub WiFi.

5.2 Perspektywy rozwoju

System umożliwia bardzo szerokie możliwości dalszego rozwoju zarówno programistycznych jak i wizualnych.

Rozwój interfejsu graficznego

Zaprojektowany w minimalny sposób interfejs graficzny można rozwijać w zasadzie bez końca. Wykorzystując dodatkową pamięć EEPROM w zegarze czasu rzeczywistego, możliwe jest też przechowywanie niektórych danych po wyłączeniu urządzenia, do analizy danych w szerszych przedziałach czasu. Dostarczając dodatkowe informacje z interfejsu OBD2, możliwe jest stworzenie kolejnych okien menu.

Rozwój komunikacji poprzez interfejs OBD2

Interfejs OBD2 udostępnia aż 10 różnych serwisów, a w trybie, z którego odczytywane są dane jak wykorzystana prędkość, obroty i przepływ powietrze, możliwe jest odczytanie 200 innych parametrów. System można w perspektywie rozwoju wykorzystać nie tylko do odczytu danych, ale także do odczytu błędów, usuwania błędów, czy testów silnika.

Zaprojektowanie uchwytu stacji z wyświetlaczem

Obudowy zaprojektowane w ramach projektu są mało funkcjonalne. Pomijając obudowę stacji OBD2, możliwe jest zaprojektowanie regulowanego uchwytu do stacji z wyświetlaczem, aby ułatwić kierowcy odczyt danych.

Inne protokoły OBD2

Stację OBD2 zaprojektowano z myślą o nowszych samochodach, w których komunikacja odbywa się za pomocą magistrali CAN. W starszych autach dość często spotykany jest protokół K-Line (ISO 9141-2 oraz ISO 14230-4). W tym celu można skorzystać np. z układu scalonego L9637D, który zamienia sygnał K-Line na UART, umożliwiając komunikację z mikrokontrolerem.

Bibliografia

- [1] Wenbo Ding. *Resources for Bluetooth Low Energy*. 2020. URL: <https://ssr-group.net/posts/2020/11/blog-post-bluetooth/>.
- [2] ehrelektronik. *OBD – On-board diagnosis*. 2019. URL: <https://ehrelektronik.com/obd/>.
- [3] Martin Falch. *OBD2 Explained - A Simple Intro [2025]*. 2025. URL: <https://www.csselectronics.com/pages/obd2-explained-simple-intro>.
- [4] Google. *Google Maps*. 2026. URL: <https://www.google.com/maps>.
- [5] Mert D Pesé, Troy Stacer, C Andrés Campos, Eric Newberry, Dongyao Chen i Kang G Shin. „LibreCAN: Automated CAN message translator”. W: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 2019, s. 2283–2300.
- [6] RFWirelessWorld. *OBD2 Frame Format and Message Structure*. 2026. URL: <https://www.rfwireless-world.com/terminology/obd2-frame-format-message-structure>.
- [7] Toyota Motor Corporation. *Aplikacja mobilna MyToyota*. 2026. URL: <https://www.toyota.eu>.

Spis rysunków

1.1	Idea systemu	4
2.1	Format ramek BLE[1]	5
2.2	Format ramek CAN[5]	6
2.3	Serwisy dostępne w ramach interfejsu OBD2[2]	7
2.4	Ramka OBD2[6]	7
2.5	Wyprowadzenia gniazda OBD2[3]	7
3.1	Schemat elektroniczny stacji OBD2	10
3.2	Wizualizacji płytki stacji OBD2	11
3.3	Schemat elektroniczny stacji z wyświetlaczem	13
3.4	Wizualizacji płytki stacji z wyświetlaczem	13
3.5	Menu stacji z wyświetlaczem	14
3.6	Modele obudów	15
3.7	Urządzenia wraz z obudowami	15
4.1	Testowanie komunikacji CAN	16
4.2	Testowanie komunikacji BLE	18
4.3	Sygnalizacja połączenia z interfejsem OBD2	18
4.4	Sygnalizacja połączenia BLE	19
4.5	Porównanie przebytej drogi	19
4.6	Porównanie przebytej drogi	19
4.7	Porównanie przebytej drogi	20
4.8	Porównanie spalania samochodu przy różnym ustawieniu przepustnicy	20
4.9	PIDy obsługiwane przez Toyota Aygo (2023)	21
4.10	Porównanie przebytej drogi	21
4.11	PIDy obsługiwane przez Skoda Kodiaq (2020)	22
4.12	Spalanie na podstoju (l/h)	22
4.13	Spalanie podczas jazdy(l/100km)	23
4.14	Spalanie podczas jazdy(l/100km)	23
4.15	Spalanie średnie na trasie (l/100km)	24